

MIP_SKILLS_CATALOG.md

Mainframe Intelligence Platform

Skills Framework

Version 1.0

Skill Structure Standard

Every skill must follow this format.

```
# Purpose

# Inputs

# Outputs

# Responsibilities

# Constraints

# Success Criteria

# Examples

# Review Checklist
```

Skill 1

repository-engineer

Purpose

Create and maintain repository structure, package layout, workspace configuration, CI/CD setup, dependency management, and engineering standards.

Inputs

- Technology stack
- Architecture requirements
- Development standards

- Repository requirements

Outputs

- Folder structure
- Python package structure
- Configuration files
- CI/CD skeleton
- Build scripts

Responsibilities

- Create repository layout
- Create package boundaries
- Define project standards
- Create reusable templates
- Maintain consistency

Constraints

- Follow Clean Architecture
- Follow SOLID principles
- Avoid circular dependencies
- Avoid unnecessary frameworks

Success Criteria

- Repository builds successfully
- Structure is maintainable
- Teams can onboard easily

Examples

- Create FastAPI service structure
- Create parser package structure
- Create testing structure

Review Checklist

- Is structure scalable?
 - Is structure maintainable?
 - Are package boundaries clear?
 - Are dependencies minimized?
-

Skill 2

mainframe-modernization-architect

Purpose

Design modernization strategy and platform architecture.

Inputs

- Business requirements
- Legacy architecture
- Modernization goals

Outputs

- Architecture decisions
- Domain models
- Roadmaps
- Modernization plans

Responsibilities

- Define architecture
- Define domain models
- Define modernization strategy

Constraints

- Modernization must be explainable
- Avoid vendor lock-in
- Preserve business logic

Success Criteria

- Architecture is scalable
- Modernization risk reduced

Examples

- Service decomposition strategy
- Knowledge graph architecture

Review Checklist

- Is architecture scalable?
 - Is modernization feasible?
 - Are risks identified?
-

Skill 3

mainframe-code-analyst

Purpose

Analyze COBOL, JCL, DB2, VSAM, IMS and CICS artifacts.

Inputs

- Source code
- Copybooks
- JCL
- SQL

Outputs

- Metadata
- Dependencies
- Lineage
- Execution flows

Responsibilities

- Extract metadata
- Discover dependencies
- Generate lineage

Constraints

- Facts only
- No assumptions
- Traceable evidence

Success Criteria

- Accurate metadata extraction
- Accurate dependency discovery

Examples

- Extract PROGRAM-ID
- Extract CALL statements
- Extract EXEC SQL

Review Checklist

- Are dependencies accurate?
- Is lineage traceable?
- Is evidence available?

Skill 4

metadata-modeler

Purpose

Design canonical metadata models.

Inputs

- Artifact metadata
- Relationship requirements

Outputs

- Entity models
- Pydantic schemas
- Metadata standards

Responsibilities

- Define entities
- Define relationships
- Define metadata lifecycle

Constraints

- Backward compatible
- Extensible

Success Criteria

- Consistent metadata model

Examples

- Program entity
- Table entity
- Relationship entity

Review Checklist

- Is model normalized?
 - Is model extensible?
 - Is naming consistent?
-

Skill 5

graph-engineer

Purpose

Design and maintain NetworkX knowledge graph.

Inputs

- Metadata entities
- Relationships

Outputs

- Graph model
- Traversal algorithms
- Query patterns

Responsibilities

- Define nodes
- Define edges
- Define graph queries

Constraints

- Graph built from facts only

Success Criteria

- Accurate graph traversal

Examples

- Program CALL graph
- Program Table lineage graph

Review Checklist

- Is relationship direction correct?
 - Are graph queries efficient?
-

Skill 6

sqlite-engineer

Purpose

Design SQLite persistence layer.

Inputs

- Metadata models
- Graph requirements

Outputs

- Database schema
- Migration scripts

Responsibilities

- Schema design
- Index strategy
- Query optimization

Constraints

- SQLite compatible
- Migration friendly

Success Criteria

- Fast metadata retrieval

Examples

- Program table
- Relationship table

Review Checklist

- Are indexes present?
 - Are duplicates prevented?
-

Skill 7

test-engineer

Purpose

Create testing strategy and automated tests.

Inputs

- Requirements
- Implementation

Outputs

- Unit tests
- Integration tests
- Coverage reports

Responsibilities

- Test design
- Edge-case validation

Constraints

- Minimum 80% coverage

Success Criteria

- Repeatable test execution

Examples

- Parser tests
- Graph tests

Review Checklist

- Are edge cases covered?
 - Are failures reproducible?
-

Skill 8

code-reviewer

Purpose

Perform architecture and code reviews.

Inputs

- Code
- Design documents

Outputs

- Findings
- Risks
- Improvements

Responsibilities

- Review quality
- Review maintainability

Constraints

- Evidence-based reviews

Success Criteria

- Reduced defects

Examples

- Parser review
- Architecture review

Review Checklist

- Is code maintainable?
 - Is code testable?
-

Skill 9

documentation-writer

Purpose

Generate project documentation.

Inputs

- Requirements
- Architecture
- Code

Outputs

- README

- Architecture docs
- User guides

Responsibilities

- Document decisions
- Document usage

Constraints

- Documentation must be versioned

Success Criteria

- New developers can onboard quickly

Examples

- Setup guide
- API documentation

Review Checklist

- Is documentation complete?
- Is documentation current?

```
:::
```

And here is the recommended workspace-wide prompt library structure:

```
:::writing{variant="document" id="38195"}
```

```
# MIP_PROMPT_LIBRARY.md
```

```
# Prompt Library Structure
```

```
```text
```

```
prompts/
```

```
|— architecture/
```

```
| |— platform_architecture.md
```

```
| |— service_design.md
```

```
| |— domain_model_review.md
```

```
| |— architecture_review.md
```

```
|— discovery/
```

```
| |— repository_scan.md
```

```
| |— artifact_classification.md
```

```
| |— inventory_generation.md
```

```
|— parsing/
```

```
| |— cobol_parser.md
```

```

| | | jcl_parser.md
| | | copybook_parser.md
| | | db2_parser.md
| | | cics_parser.md
| |
| |--- metadata/
| | |--- entity_model_design.md
| | |--- metadata_review.md
| | |--- schema_evolution.md
| |
| |--- graph/
| | |--- build_call_graph.md
| | |--- build_copybook_graph.md
| | |--- build_data_lineage_graph.md
| | |--- graph_query_design.md
| |
| |--- testing/
| | |--- parser_tests.md
| | |--- integration_tests.md
| | |--- graph_tests.md
| | |--- performance_tests.md
| |
| |--- review/
| | |--- architecture_review.md
| | |--- code_review.md
| | |--- security_review.md
| | |--- scalability_review.md
| |
| |--- modernization/
| | |--- service_discovery.md
| | |--- api_discovery.md
| | |--- event_discovery.md
| | |--- modernization_recommendations.md

```

Example Prompt: discovery/repository\_scan.md

Act as a Principal Software Architect.

Design a repository discovery framework.

Requirements:

- Python 3.13
- Recursive scanning
- SQLite persistence
- Extensible classification
- Unit testing

Do not generate code.

Provide:

1. Architecture
2. Components
3. Risks
4. Scalability Considerations
5. Testing Strategy

Example Prompt: parsing/cobol\_parser.md

Act as a Senior COBOL Analyst.

Design a metadata extraction framework.

Extract:

- PROGRAM-ID
- CALL statements
- COPY statements

Ignore:

- Business Rules
- SQL
- CICS

Do not generate code.

Provide:

1. Architecture
2. Parsing Strategy
3. Edge Cases
4. Testing Approach

Example Prompt: graph/build\_call\_graph.md

Act as a Knowledge Graph Architect.

Design a CALL relationship graph.

Input:

Program Metadata

CALL Relationships

Output:

NetworkX Graph

Provide:

1. Node Design

2. Edge Design
3. Traversal Queries
4. Scalability Considerations